AI Appliance Inspection

Insurance Claim Platform

Version 3.0.0 | June 2026

~7,800 lines of Python | *38 modules* | *250 MB model weights*

YOLO11s | *FastAPI* | *Streamlit* | *SQLite* | *Docker*

A production-grade Computer Vision + AI platform for insurance companies to automate appliance damage assessment, fraud detection, and claim processing.

Project Overview

Version: 3.0.0

Langua

What The System Does (End-to-End)

User Uploads Photo/Video

1. Imag

Technology Stack

#

Core AI/ML

Technology | Version/Purpose
Ultralytics YOLO11s | Primary model architecture (appliance detection, damage segmentation) 640640 input
Ultralytics YOLOv8n | Fallback appliance detector (~6.2 MB, runs on low-resource devices)
PyTorch 2.0+ | Deep learning framework MPS support on Apple Silicon, CUDA on NVIDIA GPUs
OpenCV 4.8+ | Image processing, heuristics, annotation, frame extraction
scikit-learn 1.3+ | ML utilities, clustering for damage analysis
scikit-image 0.21+ | Advanced image processing (DCT, ELA, segmentation helpers)
NumPy 1.24+ / Pandas 2.0+ | Numerical computation, data manipulation
Pillow 10.0+ | EXIF metadata reading, image format support
#

Web & API

Technology | Purpose
FastAPI | REST API backend (13 endpoints)
Uvicorn | ASGI server
Streamlit | Professional dashboard UI (6-tab navigation)
Python-Multipart | File upload handling
Pydantic | Request/response validation
CORS Middleware | Cross-origin support

#

Data & Storage

Technology | Purpose
SQLite | Claim history (`claim_history.db`), fraud hash database (`fraud_hashes.db`), monitoring database (`monitor.db`)
fpdf2 | PDF report generation with images and structured layout
PyYAML | Configuration files and repair cost rules
JSON | Report serialization and API responses
Loguru | Structured logging with file rotation
#

Infrastructure

Technology | Purpose
Docker | Containerized deployment (Python 3.10-slim)
Docker Compose | API + Dashboard orchestration
GitHub Actions | CI/CD pipeline (test lint build)
Pytest | Unit testing (19 tests, all passing)

Architecture Diagram

#

1. Configuration System (`configs/config.py` 294 lines)

Central configuration hub with every threshold exposed as a configurable constant:

MODEL

- Damage confidence: 35% per detection, 40% global filter
- Damage area: 0.1%60% of image (outside noise)
- Fraud scores: Low < 30, Medium < 50, High < 75, Critical 75
- Claim decision: Approve < 25, Manual Review < 50, High < 75, Reject 75
- Severity bands: Minor < 10%, Moderate < 30%, Major < 60%, Severe 60%
- Condition grades: A (85100), B (7084), C (5069), D (049)
- Damage type weights: crack=1.5, dent=1.0, display_lines=2.0, etc.
- Repair severity multipliers: Minor=0.5, Moderate=1.0, Major=1.8, Severe=3.0

#

2. Image Quality Validation (`services/image_quality.py` 149 lines)

Purpose: Gate inference behind quality checks prevents unreliable results.

Checks

Blur | Laplacian variance | < 80.0 | 25 points
Underexposure | Mean brightness | < 30/255 | 15 points
Overexposure | Mean brightness | > 230/255 | 15 points
Low Resolution | Dimensions | < 300300 | 20 points
JPEG Artifacts | 1616 block grid edge density | > 30% blocks | 10 points
Motion Blur | Laplacian ratio test | < 0.3 ratio | 20 points

Output: QualityResult(passed, score 0-100, issues[], guidance_text)

- Pass

- Failure returns structured guidance ("Please retake with the device held steady")

#

3. Appliance Detector (`models/appliance_detector/__init__.py` 202 lines)

Model: YOLO11s (yolo11s.pt, 18 MB) fallback YOLOv8n (yolov8n.pt, 6.2 MB)

Input: 6

- detect_single(image) returns highest-confidence prediction or None
- Confidence threshold 0.35: below this returns None "Unknown Appliance"
- Never forces a wrong class if unsure, escalates to manual review
- Tracks inference time and model version

Supported Appliances:

Tier | Ap

MVP | phone, television, laptop | crack, dent, display_lines

Phase 2 | tablet, monitor, refrigerator, washing_machine, air_conditioner, microwave | Appliance-specific (see config)

Sample output:

[

4. Damage Detector (`models/damage_detector/__init__.py` 307 lines)

Primary: YOLO bbox-based damage detection (phone_damage_best.pt 23 MB, laptop_damage_best.pt 18 MB, refrigerator_damage_best.pt 6 MB)

Fallback

- Phone/Laptop/TV: Screen, Keyboard, Hinge, Trackpad, Body, Edges, etc.
- General: Surface, Top, Bottom, Center

5. Confidence Filter global minimum 0.4 (configurable)

Heuristi

- Dent detection: Gaussian blur difference threshold contours 0.3%20% area range
- Display lines detection: HoughLinesP if 6 lines found, groups into bbox

Output:

[

5. Damage Segmentation (`models/damage_segmentation/__init__.py` 165 lines)

Model: YOLO11s-seg (yolo11s-seg.pt, 20 MB)

Purpose

- Converts masks to bounding boxes for backward compatibility
- Calculates mask pixel area for precise damage percentage
- Checks if damage is within beam/scan bounds
- Enables future heatmap visualization

#

6. Fraud Detection Engine (`services/fraud_service.py` 457 lines)

10-factor fraud scoring engine (0100) with 4 risk levels (Low/Medium/High/Critical):

Factor |

1. Error Level Analysis (ELA) | JPEG re-compression analysis | Weighted 20% of total

2. Metadata Anomalies | EXIF analysis missing camera, date, orientation; detects Photoshop, GIMP, Stable Diffusion, Midjourney, etc. | 40

3. Screenshot Detection | Edge density (< 2%), solid borders (std < 5), uniform color blocks | 30

4. AI-Generated Image | DCT frequency ratio (low/high > 50), noise analysis (std < 2.0), Laplacian variance < 5 | 30

5. Copy-Move Detection | 1616 block grid matching identical blocks at different positions | 20

6. Color Diversity | Unique pixel count / total pixels ratio (< 0.001 synthetic) | 15

7. Tampering Edges | High-pass filter contour analysis suspicious edge clusters | 10

8. Duplicate Image | Persistent perceptual hash in SQLite cross-session and same-session detection | 40

9. Resolution Mismatch | Too small (< 150px), too large (> 8000px), unusual aspect ratio | 20

10. Compression Anomalies | Laplacian var < 10 (over-compressed), 88 DCT block uniformity | 15

Duplicate detection uses a persistent SQLite database (fraud_hashes.db) storing 1616 perceptual hashes with first-seen timestamps and occurrence counts. If the same hash (or near-identical hash within

Output:

7. Severity Service (`services/severity_service.py` 153 lines)

Core Formula:

severity

panel_damage | 2.5 | Structural highest impact
display_lines | 2.0 | Display expensive repair
screen_crack | 1.8 | Screen common high-cost
crack | 1.5 | Structural compromise
rust | 1.2 | Progressive deterioration
body_damage | 1.1 | Cosmetic + structural
dent | 1.0 | Baseline cosmetic
scratch | 0.6 | Minor cosmetic
dead_pixels | 0.8 | Display but low impact

Defect Multiplier: 1.0 + (count 1) 0.15 multiple same-type damages compound severity

Severity

None | 0% | No damage
Minor | 010% | Cosmetic only
Moderate | 1030% | Repairable, still usable
Major | 3060% | Significant functional impact
Severe | 60100% | May be beyond economical repair

Condition Grade:

Grade |

A | 85100 | Excellent like new
B | 7084 | Good minor wear
C | 5069 | Fair damage present
D | 049 | Poor significant damage

#

8. Repair Cost Service (`services/repair_service.py` 119 lines)

Core Formula:

cost = b

crack	150	400
dent	100	300
display_lines	500	1,500
screen_crack	300	800
panel_damage	400	1,500
rust	200	600
scratch	50	150

Severity Multipliers:

Severity	
Minor	0.5
Moderate	1.0
Major	1.8
Severe	3.0

Confidence Factor: 0.5 + (confidence 0.5) low-confidence detections have lower cost impact

Output i

9. Claim Recommendation Engine (`services/claim_recommendation.py` 105 lines)

Weighted decision formula:

claim_s

Severity | 40% | None=0, Minor=10, Moderate=30, Major=60, Severe=80
Fraud Score | 30% | Direct from fraud engine (0100)
Condition | 20% | A=0, B=15, C=40, D=70 (inverted)
Damage Count | 10% | 0=0, 1=10, 2-3=30, 4+=50

Decision Thresholds:

Score R

024 | **APPROVE** | Fast-track, no human needed
2549 | **MANUAL_REVIEW** | Requires adjuster review
5074 | **MANUAL_REVIEW** | High-risk senior adjuster
75100 | **REJECT** | Deny claim, escalate to fraud team

Justification Generation:

- APP

- MANUAL_REVIEW: "Manual review needed because damage severity is high and fraud score is elevated."
- REJECT: "Claim recommended for rejection. Fraud indicators present. Damage severity is at maximum level."

#

10. Explainable AI (XAI) Service (`services/explain_service.py` 167 lines)

Generates 5-section natural language explanation for every inspection:

1. Appli

11. Multi-Image Inspection Service (`services/multi_image_service.py` 329 line

Purpose: Upload 26 photos from different angles single unified report.

View Cl

> 1.8 | Any | Close-up
< 0.5 | Any | Detail
Any | < 0.01 | Unknown
Bottom quarter bright (> 180 mean) | Any | Top view
Default | Any | Front view

Detection Merge Algorithm:

1. Run i

12. Monitoring & Observability (`services/monitoring.py` 208 lines)

Three-tier monitoring:

1. SQLi

- Average/max/min duration (ms)
- Success rate (%)
- Average confidence
- Error count and types

Usage in code:

with mo

13. Professional PDF Reports (`services/pdf_service.py` 267 lines)

Insurer-ready PDF structure:

1 | **Header** | Dark blue banner (RGB 25,55,109), claim ID, timestamp
1 | **Decision Badge** | Green check (APPROVE), yellow warning (MANUAL_REVIEW), red X (REJECT)
1 | **1. Appliance Details** | Type, confidence, condition score + grade
1 | **2. Damage Assessment** | Per-damage: type, location, confidence, area %
1 | **3. Severity Breakdown** | Base range, severity multiplier, total estimate
1 | **4. Fraud Analysis** | Score, risk level, all reasons
2 | **5. XAI Reasoning** | 5 sections: appliance, damage, fraud, repair, claim
2 | **6. Inspection Images** | Original + annotated
3 | **7. Claim Summary** | Decision, risk score, justification, metadata

#

14. Video Inspection (`scripts/inference.py` 285 lines)

Smart Frame Extraction:

- Strat

- Each frame runs through the full pipeline independently
- Damage persistence tracking damages that appear in 30% of frames are flagged as persistent
- Frame inconsistency detection if a frame lacks persistent damages, it's flagged as potentially fraudulent
- Best frame selection scores frames by damage_confidence 0.6 + (1 fraud) 0.4, picks the best
- Outputs annotated video and per-frame JSON summary

#

15. Streamlit Dashboard (`dashboard/app.py` 787 lines)

6-tab navigation:

Tab | Fe

Image | Upload image quality check run inspection side-by-side original/annotated confidence bars 5 detail tabs (Confidence, Damage, Repair Cost, Fraud, Claim) XAI panel Save PDF

Video | Upload video frame extraction per-frame analysis persistence tracking annotated video output summary download

Multi-Image | Upload 2-6 images view classification per-image inference IoU merge unified report per-image quality scores merged damage table

History | Claim list with stats (total, high-risk, avg fraud, avg condition) searchable table view details download PDF

Analytics | Key metrics (total inspections, avg fraud, avg claim, avg condition, avg repair) severity distribution claim risk distribution appliance distribution fraud score trends condition trends

Monitor | Total calls, error rate, module timing table, session stats, recent errors

Professional UI Elements:

- Decis

- Colored severity indicator dots
- Confidence bars (green 70%, yellow 4069%, red < 40%)
- Explanation boxes with blue left border
- Metric cards with background shading
- Responsive multi-column layouts

#

16. FastAPI Backend (`api/__init__.py` 330 lines)

13 API Endpoints:

Method

GET | `/` | Project info | No
GET | `/health` | Health check | No
GET | `/api/v1/info` | API metadata | No
POST | `/api/v1/quality` | Image quality check | No
POST | `/api/v1/inspect/image` | Single image inspection | API key
POST | `/api/v1/inspect/multi` | Multi-image (26) inspection | API key
POST | `/api/v1/inspect/video` | Video inspection | API key
POST | `/api/v1/fraud/advanced` | Fraud analysis only | No
POST | `/api/v1/severity` | Severity computation | No
GET | `/api/v1/claims` | List claims | No
GET | `/api/v1/claims/{id}` | Get claim details | No
GET | `/api/v1/claims/{id}/pdf` | Download claim PDF | No
GET | `/api/v1/monitor/stats` | Monitoring stats | No

#

17. Claim History (`services/claim_service.py` 150 lines)

SQLite schema (data/claim_history.db):

```
CREAT
```

18. Fraud Detection Base (`fraud_detection/__init__.py` 129 lines)

Base classes used by the advanced fraud engine:

- ELAD

- MetadataAnalyzer: EXIF extraction via Pillow TAGS mapping camera model, date, software detection
- FraudDetectionEngine: Combines ELA + metadata into unified analysis

#

19. Risk Engine (`risk_engine/__init__.py` 164 lines)

Legacy risk engine providing backward compatibility:

- RiskA

- DamageSeverityEstimator: Original area-based severity calculation
- RepairCostEstimator: Original cost rules from REPAIR_COST_RULES config
- RiskEngine: Orchestrates assessment using weighted risk formula

#

20. Utils (`utils/__init__.py` 467 lines)

Comprehensive utility library:

- Imag

- CV Operations: calculate_iou, non_max_suppression
- Class Mapping: get_appliance_from_class_id, get_class_id_from_appliance
- File Validation: validate_image_file (PIL verify), validate_video_file (OpenCV check)
- Config: load_config, save_config (YAML)
- Data: load_json, save_json
- Helpers: setup_logging, AverageMeter, format_size
- Device Detection: get_device auto-selects CUDA / MPS / CPU

Model Files

File	Size	Architecture	Purpose
`yolo11s.pt`	18 MB	YOLO11s	Primary appliance detector
`yolo11s-seg.pt`	20 MB	YOLO11s-seg	Damage segmentation (polygons)
`yolov8n.pt`	6.2 MB	YOLOv8n	Fallback appliance detector
`phone_damage_best.pt`	23 MB	YOLO11	Phone damage detection
`laptop_damage_best.pt`	18 MB	YOLO11	Laptop damage detection
`refrigerator_damage_best.pt`	6 MB	YOLO11	Refrigerator damage detection
Training checkpoints	~150 MB total	Various	Epoch checkpoints across runs

Device Support: CUDA (NVIDIA), MPS (Apple Silicon M1-M4), CPU (any)

Training Pipeline

Three dedicated training scripts:

Script |

`train_appliance_detector.py` | Appliance detection | phone, television, laptop | `datasets/appliance_detector/{train,val,test}`
`train_damage_detector.py` | Damage detection | crack, dent, display_lines | `datasets/damage_detector/{appliance}/{train,val,test}`
`train_missing_part_detector.py` | Missing parts | Appliance-specific | Future

Training Configuration:

epochs:

#

Docker

```
FROM python:3.10-slim
```

Installs

Docker Compose

```
services:
  api:  #
```

CI/CD (GitHub Actions)

```
jobs:
  test:  #
```

API Authentication

Optional API key auth via X-API-Key header. Configured via API_KEYS list in api/__init__.py.

Key Design Decisions

#

1. Never Force Wrong Classification

The appliance detector returns None when confidence < 35% instead of forcing the best-guess class. This prevents the classic MVP mistake of confidently misclassifying a refrigerator as a television. "

#

2. No Fabricated Damage

When the damage detector finds no detections above threshold, it returns an empty list. The pipeline then correctly reports "No Damage Detected" with condition score 100/100 and grade A. Previously, e

#

3. Damage Type Weights Instead of Hard Rules

Severity uses continuous weighted scoring (crack=1.5, dent=1.0, display_lines=2.0) rather than lookup tables. This generalizes to new damage types without code changes just add a weight.

#

4. Explainability as a First-Class Feature

Every decision has a corresponding natural-language explanation. This is critical for insurance: adjusters need to understand why a claim was approved/reviewed/rejected, not just the outcome.

#

5. Multi-Layer Fraud Detection

A single high ELA score doesn't trigger rejection—it's combined with 9 other factors. This reduces false positives while catching sophisticated fraud (AI-generated images, copy-move, screenshots of g

#

6. Quality Gate Before Inference

Rather than letting blurry/overexposed images through the pipeline and getting unreliable results, the quality check rejects them upfront with actionable guidance. This dramatically improves trust in

#

7. Persistent Duplicate Detection

Fraud hashes persist in SQLite across sessions. If someone uploads the same photo to two different claims weeks apart, the system detects it. This is essential for production insurance fraud detection

Test Coverage (19 tests, all passing)

Test Class	Tests	Coverage
`TestUtils`	8	IoU, NMS, resize, normalize/denormalize, class mapping
`TestFraudDetection`	3	ELA, MetadataAnalyzer, FraudDetectionEngine imports
`TestRiskEngine`	3	RiskEngine, DamageSeverityEstimator, RepairCostEstimator
`TestReportEngine`	2	InspectionReport creation, serialization
`TestModels`	2	ApplianceDetector, DamageDetector imports
`TestMissingPartDetector`	1	MissingPartDetector import

Performance Characteristics

Module | Avg Time | Notes
Image Quality Check | < 10 ms | Pure OpenCV operations
Appliance Detection | 50150 ms | YOLO11s on MPS/CUDA
Damage Detection | 50200 ms | YOLO + heuristics
Fraud Detection | 200500 ms | ELA is most expensive
Severity + Repair | < 5 ms | Pure math operations
Full Pipeline | 300900 ms | Single image, MPS/CUDA

Limitations & Known Issues

1. Phone crack model detects entire phone instead of crack region this is a dataset labeling issue, not code-fixable

2. Fridg

Quick Start

Install

pip insta